SMART HOSTEL AND ROOM ALLOCATION SYSTEM

NAME-SOUJASH CHATTERJEE

STREAM-CSE(IOTCSBT)

ENROLLMENT NO-12024052017016

ROLL N0-48

YEAR-2ND

SEMESTER-4

SESSION-SPRING 2026

OOPs PROJECT REPORT

PROBLEM STATEMENT

Manual hostel room allocation systems are widely used in many institutions, but they often suffer from inefficiencies and lack of accuracy. These systems typically rely on manual record keeping, which increases the chances of errors such as duplicate entries, incorrect room assignments, and overbooking of rooms. Additionally, manual systems lack transparency and are difficult to maintain, especially when the number of students increases.

The objective of this project is to design and implement a Smart Hostel & Room Allocation System using Object-Oriented Programming principles. The system aims to automate student registration and room allocation while ensuring that constraints such as room capacity are strictly followed. By introducing a structured and modular design, the system improves efficiency, reduces human errors, and provides a scalable solution that can be extended in the future.

SYSTEM DESIGN

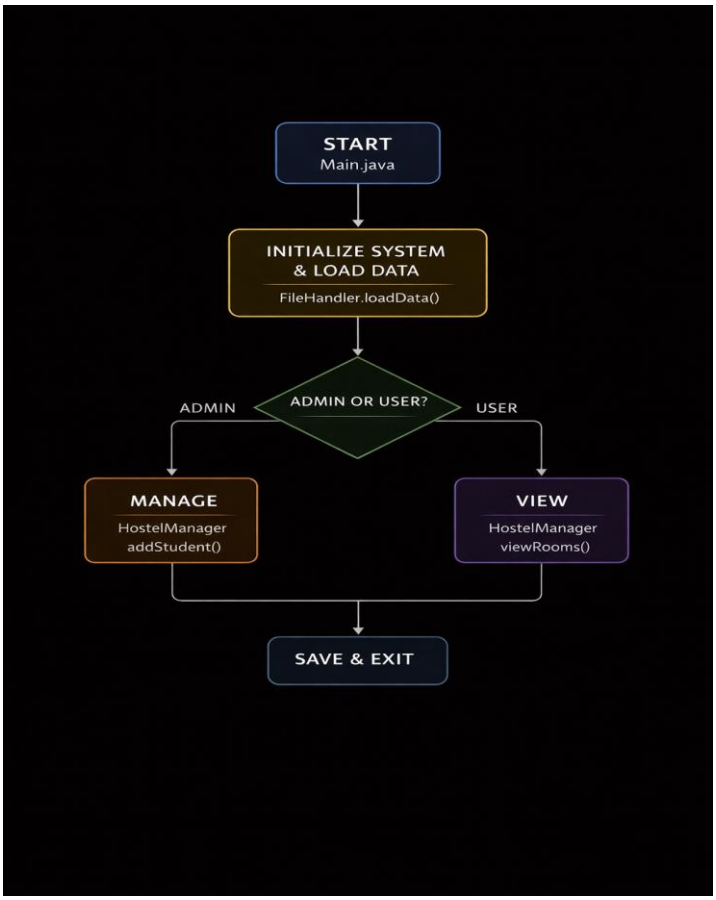
The system is designed using a modular and object-oriented approach, where responsibilities are clearly divided among different classes. The architecture follows a layered design consisting of model and controller components. The model layer includes classes such as Student and Room, which represent real-world entities, while the controller layer manages user interaction and system flow.

The major classes in the system include Student, Room, and HostelManagementSystem. The Student class stores student-related information such as ID and name. The Room class manages room details including room number, capacity, and the list of students assigned to it. It also contains important methods such as addStudent() and isFull() to handle allocation logic. The HostelManagementSystem class acts as the controller, handling user input, displaying menus, and coordinating between different objects.

The package structure is organized logically to improve readability and maintainability. Classes are separated into different files such as Student.java, Room.java, and Main.java. This separation

ensures that each class has a single responsibility and makes the system easier to extend.

The interaction flow begins when the user starts the program and selects an option from the menu. Based on the input, the system performs operations such as adding a student or allocating a room. During allocation, the system checks room capacity and assigns the student accordingly. Finally, the updated information is displayed to the user.



OOP CONCEPTS DEMONSTRATED

Encapsulation is implemented in the Student and Room classes, where data members are declared as private and accessed through public methods. This ensures that the internal state of objects cannot be modified directly, thereby improving data security and integrity.

Abstraction is applied in the room allocation process. The complexity of checking room capacity and managing student lists is hidden inside methods such as addStudent() and isFull(). The user interacts with the system without needing to understand the internal logic, making the system easier to use.

Inheritance is demonstrated through the design of room types such as SingleRoom, DoubleRoom, and TripleRoom, which can extend the base Room class. This allows common properties to be reused while enabling specific behaviors for different room types.

Polymorphism allows methods to behave differently depending on the object type. For example, allocation logic can vary based on room type, providing flexibility in system behavior.

The system follows a modular design pattern with clear separation of concerns. The controller manages system flow, while model classes handle data and logic. This improves maintainability and scalability.

IMPLEMENTATION HIGHLIGHTS

- SMART ROOM ALLOCATION –

```
27
28  for (Floor f : floors) {
29      for (Room r : f.rooms) {
30
31          if (s.getPreference().equalsIgnoreCase("Single") && r instanceof SingleRoom && !r.isFull())
32              preferredRooms.add(r);
33
34          else if (s.getPreference().equalsIgnoreCase("Double") && r instanceof DoubleRoom && !r.isFull())
35              preferredRooms.add(r);
36
37          else if (s.getPreference().equalsIgnoreCase("Triple") && r instanceof TripleRoom && !r.isFull())
38              preferredRooms.add(r);
39      }
40  }
```

This demonstrates intelligent allocation logic using user preference, type checking (instanceof), and capacity constraints, ensuring efficient and realistic room assignment.

- FILE HANDLING –

```
5
6   static void saveStudents(ArrayList<Student> students) {
7       try {
8           BufferedWriter bw = new BufferedWriter(new FileWriter("students.txt"));
9
10          for (Student s : students) {
11              bw.write(
12                  s.getId() + "," +
13                  s.getName() + "," +
14                  s.getDepartment() + "," +
15                  s.getPreference() + "," +
16                  s.getRoomNumber()
17              );
18              bw.newLine();
19          }
```

This ensures data persistence, allowing the system to store student and allocation data in a file, making the system usable across multiple sessions.

- UNIQUE ID VALIDATION –

```
13      boolean isDuplicateId(int id) {
14          for (Student s : students) {
15              if (s.getId() == id) {
16                  return true;
17              }
18          }
19          return false;
20      }
21
```

This enforces data integrity by ensuring each student has a unique ID, preventing duplication and maintaining consistency in the system.

- ROOM CHANGE –

```
139
140         if (newRoom == null) {
141             System.out.println("New room not found.");
142             return;
143         }
144
145         if (newRoom.isFull()) {
146             System.out.println("New room is full.");
147             return;
148         }
149
150         oldRoom.removeStudent(found);
151         newRoom.addStudent(found);
152
153         System.out.println("Room changed successfully.");
154     }
155
```

This demonstrates dynamic state management, where a student is safely removed from one room and added to another, reflecting real-world room transfer operations.

EXCEPTION HANDLING –

```
73
74          try {
75              choice = sc.nextInt();
76          } catch (Exception e) {
77              System.out.println("Invalid input!");
78              sc.next();
79              continue;
80          }
81
```

Prevents runtime crashes and ensures robust user input handling.

TESTING & ERROR HANDLING

Test cases considered

Normal input case: Valid student details are entered. The system successfully adds the student and allocates a room.

Multiple student allocation: Multiple students with different preferences are added. The system allocates rooms based on preference and availability.

Preference-based allocation: When a valid preference (Single/Double/Triple) is given, the system assigns a matching room if available.

Random allocation case: If the preference is invalid or unavailable, the system assigns a random available room.

Search functionality: When a valid student ID is entered, the correct student details are displayed.

Remove (vacate) functionality: When a valid student ID is entered, the student is removed and the room occupancy is updated.

Room change functionality: When a valid student ID and room number are given, the student is moved to the new room if space is available.

Role-based access: Admin has full access, while user has restricted access. Unauthorized actions are blocked.

File handling: After restarting the program, student data and room allocations are successfully restored.

Edge cases handled

Duplicate student ID: If the same ID is entered again, the system prevents duplication and asks for a new ID.

Invalid name input: If the name contains numbers or symbols, the input is rejected and the user is prompted again.

Invalid department selection: If an invalid option is chosen, the system asks the user to re-enter a valid choice.

Invalid preference input: If the preference is not Single, Double, or Triple, the system assigns a random room.

Room full condition: If all rooms are occupied, the system displays that no rooms are available.

Room change to full room: If the target room is full, the system does not allow the change.

Non-existent student search/remove: If the ID does not exist, the system displays “Student not found”.

Invalid menu input: If a non-integer or out-of-range value is entered, the system handles it and prompts again.

Failure Scenarios

Invalid input type: If a string is entered instead of a number, exception handling prevents the program from crashing and asks for input again.

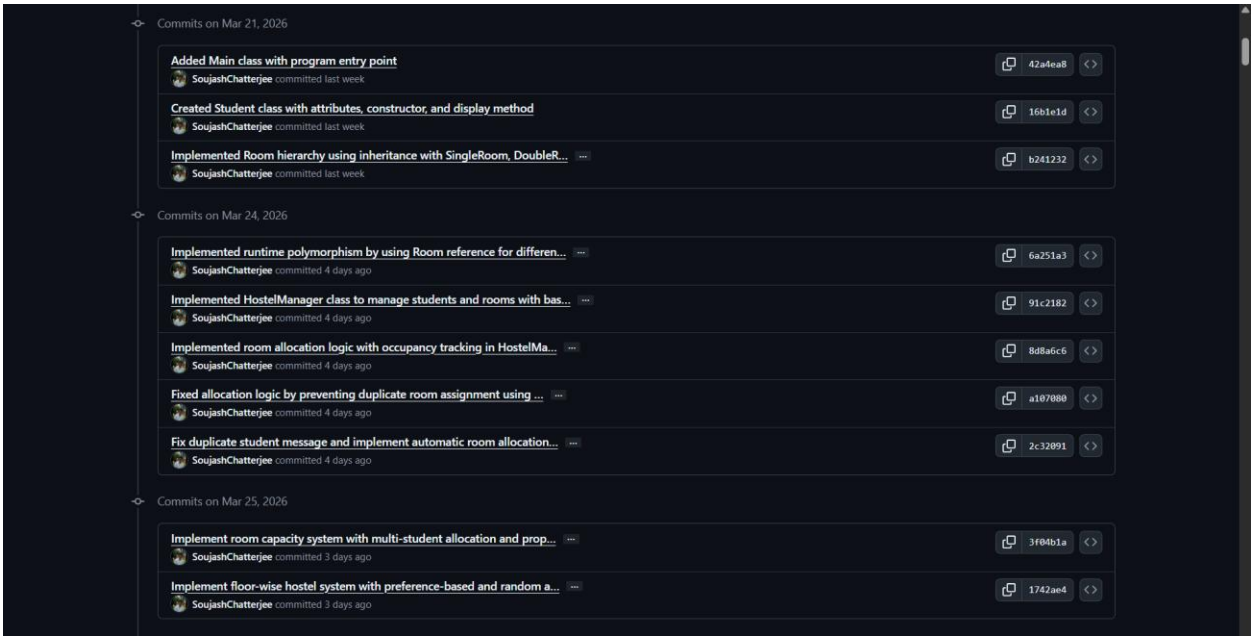
File not found (first run): If the data file does not exist, the system skips loading without crashing.

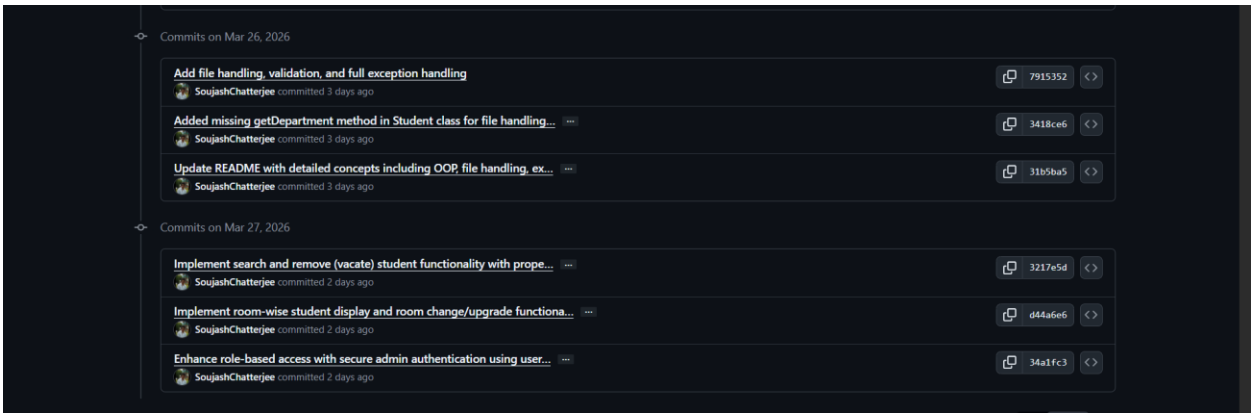
File read/write errors: If there is an I/O issue, an error message is shown and the program continues.

Unauthorized access attempt: If a user tries to access admin features, the system denies access.

Incorrect login credentials: If wrong admin credentials are entered, the system logs in as a normal user.

GIT WORKFLOW





The project was developed in a step-by-step manner, starting with the creation of the main entry point using the Main class to establish the basic execution flow and menu-driven interface. After that, the Student class was created to represent student data with attributes like name and ID along with display functionality.

Next, the room structure was implemented using object-oriented principles. A base Room class was created and extended into SingleRoom, DoubleRoom, and TripleRoom, demonstrating inheritance and polymorphism. Runtime polymorphism was further applied by handling different room types through a common reference.

The HostelManager class was then introduced to manage all core operations such as adding students, allocating rooms, and maintaining records. Initially, a basic room allocation system was implemented, which was later improved by adding occupancy tracking and fixing issues like duplicate assignments.

The system was then enhanced by introducing room capacity logic, allowing multiple students in a room based on type. A floor-wise structure was also added, organizing rooms under different floors to make the system more realistic and scalable.

Further improvements included preference-based allocation, where students could choose room types, along with a fallback mechanism that assigns rooms randomly if the preferred option is not available. This added intelligent decision-making to the system.

Validation and exception handling were implemented to ensure robustness. This included checking for duplicate student IDs, validating names and inputs, and handling incorrect data entries without crashing the program.

File handling was added to enable data persistence. Student details along with room allocations were saved to a file and reloaded when the program restarted, ensuring continuity of data.

Additional features such as searching for a student, removing (vacating) a student, displaying room-wise student details, and changing rooms were implemented to make the system more functional and closer to real-world applications.

Finally, role-based access control was introduced with a secure admin login using username and password. This restricted critical operations to authorized users only, adding a basic level of security.

Overall, the project evolved from a simple structure into a complete system with object-oriented design, intelligent allocation logic, persistent storage, validation, and security features.

CONCLUSION & FUTURE SCOPE

The Smart Hostel & Room Allocation System successfully demonstrates the application of Object-Oriented Programming concepts to solve a real-world problem. The system provides an efficient and reliable way to manage student accommodation and room allocation.

The modular design and use of OOP principles make the system easy to maintain and extend. Future enhancements can include the addition of a graphical user interface, integration with a database, and implementation of advanced authentication mechanisms. Overall, the project serves as a strong example of how structured programming and design principles can be used to build practical and scalable software solutions.